

系统开发工具基础·实验报告（第 9–12 题）

姓名：李云龙 学号：25030021043

2026 年 9 月 14 日

姓名	李云龙
学号	25030021043
实验日期	2026 年 9 月 14 日
实验环境	Windows 11 + WSL2 (Ubuntu 26.04 LTS) ; Python 3.14 + setup-tools(build) + pytest 9.1 + PyTorch 2.9.1 (CPU)

说明：本报告中各题“运行结果”均为在 WSL 中实际运行的输出；第 10 题的“失败测试”与第 12 题的损失值等关键过程均如实记录。

1 实验目的

1. 掌握 Python 包的标准打包流程（pyproject.toml、构建后端、入口点），并在干净虚拟环境中验证 wheel 的可安装性。
2. 掌握“测试驱动 + 编程智能体”的可验证修复循环，以及人工 diff 审查的必要性。
3. 掌握把模糊的协作材料（Issue / 提交信息 / 评审意见）改写成可执行信息的方法。
4. 掌握 PyTorch 线性回归训练循环的完整写法（梯度清零、反向传播、参数更新、评估模式）。

2 实验九：从源码构建并在干净环境安装 Wheel（q09）

2.1 包结构与声明

pyproject.toml:

```
[build-system]
requires = ["setuptools>=68"]
build-backend = "setuptools.build_meta"

[project]
name = "greetlab-25030021043"
version = "0.1.0"

[project.scripts]
sdt-greet = "greetlab.cli:main"
```

src/greetlab/cli.py:

```
import argparse

def main():
    p = argparse.ArgumentParser()
    p.add_argument("--name", required=True)
    a = p.parse_args()
    print(f"Hello, {a.name}!")
```

(src/greetlab/__init__.py 为包标记文件。)

2.2 构建与安装步骤

```
cd q09
python3 -m build                                # 1. 从源码构建
python3 -m venv /tmp/greetenv                    # 2. 新建干净虚拟环境
/tmp/greetenv/bin/pip install dist/*.whl         # 3. 只从 wheel 安装
cd /tmp && /tmp/greetenv/bin/sdt-greet --name 25030021043 # 4. 在 q09 之外运行
```

2.3 运行结果

构建产物：

```
dist/greetlab_25030021043-0.1.0-py3-none-any.whl
dist/greetlab_25030021043-0.1.0.tar.gz
```

干净 venv 中只装 wheel 后：

```
greetlab-25030021043 0.1.0
```

在 q09 目录之外运行：

```
$ sdt-greet --name 25030021043
Hello, 25030021043!
```

2.4 要点分析

- pyproject.toml 是标准打包声明：[build-system] 指定构建后端 (setuptools)，[project] 提供元数据，[project.scripts] 把 greetlab.cli:main 注册为命令行入口 sdt-greet；
- 干净 venv + 只装 wheel 模拟真实用户的安装路径，验证 wheel 自包含（不依赖源码目录、不依赖开发环境），这是发布前的“发货检查”；
- wheel 文件名体现了分发名（连字符归一为下划线）、版本、Python 标签 (py3) 与平台标签 (none-any = 纯 Python 跨平台)。

3 实验十：让编程智能体进入可验证的修复循环 (q10)

3.1 失败的测试 (先写测试)

tests/test_cli.py:

```
import sys

import pytest

from greetlab.cli import main

@pytest.mark.parametrize("blank", [" ", "   ", "\t"])
def test_blank_name_exits_with_code_2(monkeypatch, blank):
    monkeypatch.setattr(sys, "argv", ["sdt-greet", "--name", blank])
    with pytest.raises(SystemExit) as excinfo:
        main()
    assert excinfo.value.code == 2
```

3.2 修复前：测试失败

```
$ python3 -m pytest -q
tests/test_cli.py:11: Failed
----- Captured stdout call -----
Hello,  !
FAILED tests/test_cli.py::test_blank_name_exits_with_code_2[ ]
FAILED tests/test_cli.py::test_blank_name_exits_with_code_2[   ]
FAILED tests/test_cli.py::test_blank_name_exits_with_code_2[\t]
3 failed in 0.47s
```

3.3 智能体改动 (最小修复)

```
def main():
    p = argparse.ArgumentParser()
    p.add_argument("--name", required=True)
    a = p.parse_args()
    if not a.name.strip():
        p.error("--name must not be blank")  # argparse 的 error 以 SystemExit(2) 结束
    print(f"Hello, {a.name}!")
```

3.4 修复后：测试通过

```
$ python3 -m pytest -q
...
3 passed in 0.33s [100%]
```

3.5 ai_log.md（核心提示 / 智能体改动 / 人工验证）

1. 核心提示: `sdt-greet --name " "`（纯空白）当前仍输出问候语；目标=main 以 `SystemExit(2)` 结束；
约束=不改 CLI 语法、不引入依赖；验证命令=`python3 -m pytest -q`
2. 智能体改动: 仅改 `src/greetlab/cli.py`, `print` 前加 `if not a.name.strip(): p.error(...)`
3. 测试演进: 修复前 3 个参数化用例失败, 修复后 3 passed (红 -> 绿)
4. 人工验证: `diff` 仅新增 3 行无无关修改; `pytest` 复跑通过

3.6 要点分析

- 这是 TDD + 智能体闭环：先用失败测试把“目标”变成可验证的断言（`SystemExit(2)`），再让智能体改实现，最后用同一条命令判定成败；
- 约束要显式给出（不改语法、不加依赖），否则智能体可能“顺手”重构出无关改动；
- 人工 `diff` 审查是闭环的最后一道闸门：确认改动最小、无副作用后再合并。

4 实验十一：把“无法处理”的协作材料改成可执行信息（q11）

4.1 Issue（重写）

- 标题: `sdt-greet` 对空白 `--name` 未拒绝, 仍输出问候语
- 环境: Linux/WSL, Python 3.14, wheel 安装的 `greetlab-25030021043`; Windows 上是否复现: 待确认
- 复现命令: `sdt-greet --name " "`
- 期望结果: 拒绝空白姓名, 以非零退出码结束
- 实际结果: 输出 `Hello, !`, 退出码 0

4.2 提交信息（重写）

标题: Reject blank `--name` in greetlab CLI

正文: 传入纯空白字符串时程序仍打印问候语并以 0 退出, 会掩盖调用方的参数错误。在 main 中对 `a.name` 做 `strip` 校验, 空白时调用 `parser.error` 以 `SystemExit(2)` 拒绝, 并补充参数化回归测试。

4.3 评审意见（重写, 标注级别）

[Suggestion] 行为修复正确, 但校验放在 main 内、与解析逻辑分离。建议把校验抽成 `--name` 的 `type` 函数（如 `not_blank`）, 在参数解析阶段即拒绝, 逻辑更内聚、便于直接单测。风险点: 现分支已满足要求, 但后续新增入口需复用该校验, 抽离可避免重复。建议动作: 抽取 `type` 函数并补参数化测试, 下个 PR 完成, 不阻塞本次合并。

（全文约 340 字, 满足 400 字以内要求。）

4.4 要点分析

- 原始材料“不可执行”：Windows 上运行不了 缺环境与复现步骤, fix bug 无信息量, 写得不好, 重写无可操作动作;
- 重写标准是**接手人能否直接照做**：Issue 给环境 + 复现 + 期望/实际, 未知项显式标注“待确认”; 提交信息用祈使句标题 + 问题与方案正文; 评审意见给出具体行为、风险、建议动作并标注 Blocking/Suggestion/Nit 级别。

5 实验十二：修复一个可复现的线性回归训练循环（q12）

5.1 补全后的代码（q12/train.py）

```
import torch
from torch import nn

torch.manual_seed(20260907)

x = torch.linspace(-1, 1, 101).reshape(-1, 1)
y = 3 * x - 1

model = nn.Linear(1, 1)
loss_fn = nn.MSELoss()
opt = torch.optim.SGD(model.parameters(), lr=0.1)

for _ in range(200):
    pred = model(x)
    loss = loss_fn(pred, y)
    opt.zero_grad()    # 清空上一轮梯度
    loss.backward()    # 反向传播
    opt.step()         # 更新参数

model.eval()          # 评估模式
with torch.no_grad(): # 关闭梯度跟踪
    final_loss = loss_fn(model(x), y)
    w = model.weight.item()
    b = model.bias.item()

print(f"final loss = {final_loss.item():.8f}")
print(f"weight = {w:.6f}")
print(f"bias = {b:.6f}")

assert final_loss.item() < 0.001, "final loss 应小于 0.001"
print("OK: final loss < 0.001")
```

5.2 运行结果

```
$ python3 train.py
```

```
final loss = 0.00000000
weight = 2.999997
bias = -1.000000
OK: final loss < 0.001
```

5.3 要点分析

- 训练循环三要素缺一不可：`zero_grad()` 清空上一轮梯度（否则梯度会累积）、`backward()` 反向传播求梯度、`step()` 按优化器规则更新参数；
- 评估阶段用 `model.eval() + torch.no_grad()`：前者切换评估态（影响 Dropout/BatchNorm 等），后者不构建计算图，省内存也更快；
- 最终 `weight ≈ 3`、`bias ≈ -1` 是 200 轮梯度下降**训练收敛**的结果（`torch.manual_seed(20260907)` 保证可复现），并非直接赋值；最终损失 $0 < 0.001$ ，满足要求。

6 遇到的问题与解决

问题	原因	解决
空白姓名未报错	<code>argparse</code> 只校验参数是否存在，不校验内容	在 <code>main</code> 中 <code>strip</code> 后判空并调用 <code>parser.error</code> （退出码 2）
测试导入不到 <code>src</code> 布局的包	<code>src</code> 布局下包不在 <code>sys.path</code>	增加 <code>conftest.py</code> 把 <code>src</code> 加入 <code>sys.path</code>
环境缺少 PyTorch	Ubuntu 26.04 默认未安装	<code>apt install python3-torch</code> (CPU 版 2.9.1)
智能体可能引入无关改动	缺少显式约束	提示中写明约束（不改语法/不加依赖），并人工 <code>diff</code> 审查

7 实验总结

通过本次实验，我完整走通了一个 Python 包从源码到 wheel 再到干净环境安装的发布链路，理解了 `pyproject.toml` 中构建后端与入口点的作用；实践了“失败测试 -> 智能体修复 -> 人工验证”的可验证修复循环，体会到明确目标、显式约束与 `diff` 审查三者缺一不可；学会了把模糊的协作材料改写成含环境、复现步骤、期望/实际结果的可执行信息；并补全了 PyTorch 线性回归的完整训练循环，掌握了梯度清零/反传/更新与评估模式的正确用法，最终损失收敛到 0（小于 0.001）。

附录：文件清单

<code>w3/README.md</code>	第 9--12 题说明（步骤、原理、实测数据）
<code>w3/q09/pyproject.toml</code>	第 9 题打包声明（包名 <code>greetlab-25030021043</code> ）
<code>w3/q09/src/greetlab/</code>	第 9 题包源码（ <code>__init__.py</code> 、 <code>cli.py</code> ）
<code>w3/q09/dist/</code>	第 9 题构建产物（ <code>.whl</code> 与 <code>.tar.gz</code> ）
<code>w3/q10/src/greetlab/cli.py</code>	第 10 题修复后的实现（空白校验）

w3/q10/tests/test_cli.py	第 10 题失败测试 (参数化空白名)
w3/q10/conftest.py	第 10 题测试导入配置
w3/q10/ai_log.md	第 10 题智能体循环记录 (<=5 行)
w3/q11/communication.md	第 11 题协作材料改写 (<=400 字)
w3/q12/train.py	第 12 题 PyTorch 线性回归训练
实验报告.md / 实验报告.tex / 实验报告.pdf	本报告

<https://github.com/lll0178/sys-dev-tools-lab>

